

1. 시저 암호와 비제네르 암호를 설명하고, 두 암호의 차이점을 기술하시오.**1-가. 시저 암호와 비제네르 암호의 암호화 방법을 설명하시오.**

시저 암호는 단일 알파벳 치환 암호이고, 비제네르 암호는 다중 알파벳 치환 암호이다. 시저 암호는 단일 치환 테이블을 이용하여 평문의 모든 문자에 동일한 변환을 적용한다. 변환은 26가지로 구분되는데 각 변환을 알파벳 순서에 대한 이동값 0~25로 표현하는 것이 가능하다. 이동값이 3으로 정해졌다면 A는 D로, B는 E로, X는 A로 바뀐다. 경우의 수가 26이기 때문에 전수조사에 취약하다.

비제네르 암호는 다중 알파벳 치환 암호이므로 여러 개의 치환 테이블을 이용한다. 키 길이는 단어를 통해 주어지는데 이 키 길이는 보통 평문보다 짧게 구성된다. 그리고 키 전체를 반복해서 이어붙여 문자열과 동일한 길이를 만들고 각 문자가 각 키의 문자 위치에 의한 시저 변환을 진행한다. 즉, 주기성을 가진 다중 알파벳 치환 암호가 된다.

1-나. 시저 암호와 비제네르 암호를 이용한 암호문의 알파벳 빈도 분포 차이점을 서술하시오

시저 암호는 단일 문자 치환 암호이므로 알파벳 빈도 분포표의 문자별 차이만 발생할 뿐 전체 분포는 그대로 유지된다. 즉, 임의의 영어 문자열로부터 빈도 분포를 얻는 것과 같다.

비제네르 암호는 주기성을 갖는 다중 알파벳 치환 암호이므로 전체 암호문에 대한 알파벳 빈도 분포는 단일 알파벳 문자 치환 암호와 다르게 된다. 하지만 키 길이를 안다고 할때 동일 키가 작용하는 위치별 문자를 모아 분포를 구하면 단일 치환 암호에서 분포를 조사한 것과 같은 결과를 얻게 된다.

2. “ciphertext1.txt”, “ciphertext2.txt” 각각에 대해 영어 자연어와의 일치지수 (Index of Coincidence, IC)를 계산하고, 이를 이용하여 암호문을 시저 암호와 비제네르 암호로 구분하시오.

일치지수는 주어진 문자열에서 무작위로 선택된 두 문자가 일치할 확률이다. 각 문자가 나타날 확률의 제곱합을 구하면 문제에서 제시된 빈도표에 의해서 이론적 값은 0.0655가 된다.

치환 암호에서 단일 치환 암호는 문자의 고유한 빈도 분포를 흐트러뜨리지 않지만, 암호문으로부터 쉽게 문자의 빈도수를 알기는 어렵다. 이런 어려움을 해소하기 위해 문자열로부터 일치지수를 계산하는 공식도 존재한다.

길이가 N 인 주어진 문자열에서 임의의 두 문자를 선택하는 경우의 수는 $\frac{N(N-1)}{2}$ 이다. 그리고 문자열에 있는 i 개 문자가 있을 때 각 문자가 나타난 개수를 세어 n_i 로 표기한다. n_i 의 총합은 문자열의 길이 N 이 된다. i 번째 문자만 포함하는 쌍의 개수는 $\frac{n_i(n_i-1)}{2}$ 가 되고 무작위로 선택된 문자가 일치하는 경우의 수는 $\sum_A^Z \frac{n_i(n_i-1)}{2}$ 이 된다. 일치지수는 무작위로 선택된 두 문자가 일치할 확률을 나타내므로, 수학적 확률을 계산하면 일치지수는 $\frac{\sum_A^Z n_i(n_i-1)}{N(N-1)}$ 이 된다. 암호문의 일치지수는 단일 치환암호라면 이론적 값인 0.066에 가까운 값이 나타날 것이다.

2-가. “ciphertext1.txt”, “ciphertext2.txt”의 일치지수를 각각 계산하시오. 이때, 일치지수는 소수 넷째자리에서 반올림하여 소수점 셋째자리 까지 표기한다.

주어진 문자열로부터 일치지수를 계산하는 코드는 다음과 같다. 주어진 파일에서 문자는 대문자만 사용하고 있으

며, 영어 문자에 대한 단일 치환암호 특성 상 줄 바꿈이나 공백 등 특수 문자에 대한 암호화는 진행하지 않는다.

```
import string
upper = string.ascii_uppercase
def read_cipher(filename):
    with open(filename, "r") as f:
        return f.read().replace("\n", "").strip().upper()
def IC(text):
    N = len(text)
    if N < 2:
        return 0
    counts = [text.count(c) for c in upper]
    numerator = sum(n * (n - 1) for n in counts)
    denominator = N * (N - 1)
    return numerator / denominator

ciphertext1 = read_cipher("ciphertexts1.txt")
ciphertext2 = read_cipher("ciphertexts2.txt")

print(f"Ciphertext1 IC: {IC(ciphertext1):.3f}")
print(f"Ciphertext2 IC: {IC(ciphertext2):.3f}")
```

계산된 일치지수는 다음과 같다. 암호문 1: 0.068, 암호문 2: 0.044

2-나. 계산된 일치지수 값을 이용하여 각 암호문의 암호 방식을 특정하고 그 근거를 설명하시오

계산된 일치지수는 Ciphertext1 0.068, Ciphertext2 0.044으로, Ciphertext1이 영어 자연어의 일치지수 0.066에 가까운 값이 된다 즉, 각 알파벳이 나타난 빈도의 분포가 비슷하므로 단일 치환암호임을 확인할 수 있다.

3. 시저 암호로 분류된 암호문의 평문을 복원하여라.

3-가. '2번' 문제에서 시저 암호로 분류된 암호문에 대하여, 가능한 모든 키 값을 이용하여 암호문을 복호화하시오. 그리고 각 키로 복호화된 결과에서 알파벳(A-Z)별 출현 빈도를 계산하시오.

가능한 경우의 수는 26가지로 복호화 결과로부터의 분포는 다음과 같이 나타난다. 단일치환암호이므로, 모든 암호문을 묶어 측정되는 도수를 늘리고자 했다.

복호화 키	암호문의 알파벳별 출현 빈도(총 19,897개)												
0	A	B	C	D	E	F	G	H	I	J	K	L	M
	428	181	134	45	355	57	1,676	459	751	734	2,524	495	245
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	899	1,484	37	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905
	A	B	C	D	E	F	G	H	I	J	K	L	M
	181	134	45	355	57	1,676	459	751	734	2,524	495	245	899
2	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	1,484	37	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428
	A	B	C	D	E	F	G	H	I	J	K	L	M
	134	45	355	57	1,676	459	751	734	2,524	495	245	899	1,484
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	1,484	37	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428

	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>37</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	37	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
37	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181															
3	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	45	355	57	1,676	459	751	734	2,524	495	245	899	1,484	134
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	45	355	57	1,676	459	751	734	2,524	495	245	899	1,484	134														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37															
4	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	355	57	1,676	459	751	734	2,524	495	245	899	1,484	134	36
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	355	57	1,676	459	751	734	2,524	495	245	899	1,484	134	36														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45															
5	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	57	1,676	459	751	734	2,524	495	245	899	1,484	134	36	1,016
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	57	1,676	459	751	734	2,524	495	245	899	1,484	134	36	1,016														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355															
6	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	1,676	459	751	734	2,524	495	245	899	1,484	134	36	1,016	499
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	1,676	459	751	734	2,524	495	245	899	1,484	134	36	1,016	499														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355	57
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355	57															
7	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	459	751	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	459	751	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
1,479	444	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676															
8	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	751	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	751	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	444	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
444	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459															
9	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>734</td><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459	751
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
28	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459	751															
10	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>2,524</td><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>1,198</td><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459	751	734
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
1,198	1,361	1,905	428	181	37	45	355	57	1,676	459	751	734															
11	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>495</td><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>1,361</td><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	1,361	1,905	428	181	37	45	355	57	1,676	459	751	734	2,524
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
1,361	1,905	428	181	37	45	355	57	1,676	459	751	734	2,524															
12	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>245</td><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>1,905</td><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	1,905	428	181	37	45	355	57	1,676	459	751	734	2,524	495
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
1,905	428	181	37	45	355	57	1,676	459	751	734	2,524	495															
13	<table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td><td>H</td><td>I</td><td>J</td><td>K</td><td>L</td><td>M</td></tr><tr><td>899</td><td>1,484</td><td>134</td><td>36</td><td>1,016</td><td>499</td><td>1,427</td><td>1,479</td><td>444</td><td>28</td><td>1,198</td><td>1,361</td><td>1,905</td></tr></table>	A	B	C	D	E	F	G	H	I	J	K	L	M	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905
	A	B	C	D	E	F	G	H	I	J	K	L	M														
	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905														
	<table><tr><td>N</td><td>O</td><td>P</td><td>Q</td><td>R</td><td>S</td><td>T</td><td>U</td><td>V</td><td>W</td><td>X</td><td>Y</td><td>Z</td></tr><tr><td>428</td><td>181</td><td>37</td><td>45</td><td>355</td><td>57</td><td>1,676</td><td>459</td><td>751</td><td>734</td><td>2,524</td><td>495</td><td>245</td></tr></table>	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	428	181	37	45	355	57	1,676	459	751	734	2,524	495	245
N	O	P	Q	R	S	T	U	V	W	X	Y	Z															
428	181	37	45	355	57	1,676	459	751	734	2,524	495	245															

14	A	B	C	D	E	F	G	H	I	J	K	L	M
	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
15	181	37	45	355	57	1,676	459	751	734	2,524	495	245	899
	A	B	C	D	E	F	G	H	I	J	K	L	M
	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181
16	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	37	45	355	57	1,676	459	751	734	2,524	495	245	899	1,484
	A	B	C	D	E	F	G	H	I	J	K	L	M
17	36	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	45	355	57	1,676	459	751	734	2,524	495	245	899	1,484	134
18	A	B	C	D	E	F	G	H	I	J	K	L	M
	1,016	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
19	355	57	1,676	459	751	734	2,524	495	245	899	1,484	134	36
	A	B	C	D	E	F	G	H	I	J	K	L	M
	499	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355
20	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	57	1,676	459	751	734	2,524	495	245	899	1,484	134	36	1,016
	A	B	C	D	E	F	G	H	I	J	K	L	M
21	1,427	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355	57
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	1,676	459	751	734	2,524	495	245	899	1,484	134	36	1,016	499
22	A	B	C	D	E	F	G	H	I	J	K	L	M
	1,479	444	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
23	459	751	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427
	A	B	C	D	E	F	G	H	I	J	K	L	M
	444	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459
24	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	751	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479
	A	B	C	D	E	F	G	H	I	J	K	L	M
25	28	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459	751
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	734	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444
26	A	B	C	D	E	F	G	H	I	J	K	L	M
	1,198	1,361	1,905	428	181	37	45	355	57	1,676	459	751	734
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
27	2,524	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28
	A	B	C	D	E	F	G	H	I	J	K	L	M
	1,361	1,905	428	181	37	45	355	57	1,676	459	751	734	2,524
28	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	495	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198
	A	B	C	D	E	F	G	H	I	J	K	L	M
29	1,905	428	181	37	45	355	57	1,676	459	751	734	2,524	495
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	1,905	428	181	37	45	355	57	1,676	459	751	734	2,524	495

	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	245	899	1,484	134	36	1,016	499	1,427	1,479	444	28	1,198	1,361

카이사르 암호 특성 상 한 암호문의 출현 개수를 작성하고 나면 이것을 밀어서 다음 키의 출현 빈도로 작성하는 것이 가능하다.

3-나. [표 1]에서 제시된 영어 알파벳 출현 빈도(기대값)를 기준으로, '3-가'에서 구한 알파벳별 출현 빈도에 대하여 카이제곱(Chi-squared) 적합도 검정을 수행하시오. 그리고, 해당 값을 이용하여 키로 추측되는 값을 결정하시오.

아래 코드를 실행해서 결과를 얻는다.

```
import string
alphabet = string.ascii_uppercase
freq = [
    8.17, 1.50, 2.78, 4.25, 12.70, 2.23, 2.02,
    6.09, 6.97, 0.15, 0.77, 4.03, 2.41, 6.75,
    7.51, 1.93, 0.10, 5.99, 6.33, 9.06, 2.76,
    0.98, 2.36, 0.15, 1.97, 0.07
]
freq_c = [
    428, 181, 134, 45, 355, 57, 1676,
    459, 751, 734, 2524, 495, 245, 899,
    1484, 37, 36, 1016, 499, 1427, 1479,
    444, 28, 1198, 1361, 1905
]
total = sum(freq_c)
freq_n = [
    total * percentage / 100
    for percentage in freq
]
results = []
def rotlist(values, count):
    count %= len(values)
    return values[count:] + values[:count]
def chi_square(observed, expected):
    return sum(
        (observed_count - expected_count) ** 2 / expected_count
        for observed_count, expected_count in zip(observed, expected)
    )
for key in range(26):
    freq_r = rotlist(freq_c, key)
    score = chi_square(freq_r, freq_n)
    results.append((key, score))
    print(f"Key {key:2d}: chi-square = {score:.3f}")
best_key, best_score = min(results, key=lambda result: result[1])
print()
print(f"관측 문자 수: {total}")
print(f"추정 복호화 키: {best_key}")
print(f"추정 이동 문자: {alphabet[best_key]}")
print(f"최소 카이제곱 값: {best_score}")
```

Ciphertext1의 관측 도수 합계는 총 19,897로, 주어진 힌트 $\chi^2(k) = \sum_{i=1}^z \frac{(freq(P_i) - freq(N_i))^2}{freq(N_i)}$ 는 $freq(P_i)$ 를 빈도 분석 표로부터 얻고, $freq(N_i)$ 은 총 관측 도수와 문제에서 제시된 비율을 곱해서 얻는다.

freq는 비율값이며, freq_n으로 계산하기 위해 19897과 백분율 값을 곱한다. 그리고 freq_p는 복호화 키가 0인 원래 암호문의 빈도 freq_c로부터 왼쪽으로 리스트를 회전시키는 rotlist를 타고 얻는다. 주어진 공식을 통해 카이제곱 거리를 계산하면 결과는 다음과 같다.

키	A	B	C	D	E	F	G	H	I	J	K	L	M
값	370762	349321	157068	135053	185318	118928	922	290534	167610	357276	144993	564892	68513

키	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
값	240028	150544	275293	301101	100815	123254	73449	508237	199550	117261	220337	259621	176044

자연어와 통계적 거리가 가장 작은 복호화 키는 G(6)이 된다.

따라서 암호화 키는 26에서 복호화 키 위치를 뺀 값인 U(20)이 된다.

3-다. 위에서 추측한 키 값으로 다음 암호문을 복호화하고 평문을 제시하시오

복호화 키G 로 복호화하면 다음과 같다.

```
THESOVEREIGNTYOFTHEREPUBLICOFKOREASHALLRESIDEINTHEPEOPLEANDALLSTATEAUTHORITYSH
ALLEMANATEFROMTHEPEOPLE
```

4. 비제네르 암호로 분류된 암호문의 키 길이를 추정하고 실제 키와 평문을 복원하시오.

4-가. 암호문에서 3글자 이상의 반복되는 패턴을 탐색하고, 반복 패턴 간의 위치 간격을 키 길이를 추측하시오. 단, 키 길이는 5 이하이다.

암호문의 길이가 각각 다르므로, 각 줄별로 암호화를 새로 적용한 것으로 추정된다. 파이썬의 readlines를 통해 암호문을 원소로 갖는 리스트를 만들고 줄별로 암호문을 추출한다. 해당 리스트에서 반복되는 패턴이 섞였을 가장 긴 암호문을 구한다.

```
SYXOBOLKSACZICWKNOWEOXDZRDLMBOCQWLOZMOWOXKZXIPZCODSZARVTOOAIQJFLODVDXVTXS
XPQDQRKWJFXDSZFBKDKFN CNMWKOOWMJMENIBDLDBBNLHOPODBLYYISLDTJBFSECHQOAZFOYCHO
WMPJTXPQDQRKWYICSPNGDMSVGLYXWOCKNOWXXOMWUVLIRBYQJFCRSZGQKWGBXDMZSWD TOZNNEJ
OLVLDAJQLDBBDECSBDLOSXBAPPUSNJFPKYDNJDTJBXXECSPBZPBMCZAIWVLRTDVLXHBMZHARDEZRKI
APPUSNJTOSNDOUCTIHQONJIACPJTXPQDQRKWYICSPNPDDDCOUMVMZSWD TOZNNZIZHDZXCVCZPIGJDTJB
BKDKFN CNMWKOOWMJME
```

반복되는 패턴은 LPS알고리즘으로 여럿 탐지하며, 패턴 간의 간격은 KMP알고리즘으로 찾을 수 있다.

LPS알고리즘은 긴 문자열 내에 존재하는 가장 긴 Suffix와 Prefix의 길이를 찾는다. 아래 코드에서 5개의 패턴을 얻도록 했다.

KMP알고리즘은 긴 문자열 내에 존재하는 패턴 간의 간격을 얻는다.

```
def longest(filename):
    with open(filename, "r") as f:
        lines = f.readlines()

    lines = [line.strip() for line in lines]
    return max(lines, key=len)

def lps(text):
    table = [0] * len(text)
```

```
length = 0
i = 1

while i < len(text):
    if text[i] == text[length]:
        length += 1
        table[i] = length
        i += 1
    elif length > 0:
        length = table[length - 1]
    else:
        i += 1

return table
```

```
def long_patterns(text, count=5):
    patterns = set()

    for start in range(len(text)):
        suffix = text[start:]
        table = lps(suffix)

        for length in table:
            if length > 0:
                patterns.add(suffix[:length])

    patterns = sorted(patterns, key=lambda pattern: (-len(pattern), pattern))
    selected = []

    for pattern in patterns:
        if not any(pattern in longer for longer in selected):
            selected.append(pattern)

        if len(selected) == count:
            break

    return selected
```

```
def kmp(text, pattern):
    positions = []
    table = lps(pattern)
    i = 0
    j = 0

    while i < len(text):
```

```

        if text[i] == pattern[j]:
            i += 1
            j += 1

            if j == len(pattern):
                positions.append(i - j)
                j = table[j - 1]
            elif j > 0:
                j = table[j - 1]
            else:
                i += 1

    return positions

ciphertext = longest("ciphertexts2.txt")
patterns = long_patterns(ciphertext)

print(ciphertext)
print("Length:", len(ciphertext))

for rank, pattern in enumerate(patterns, 1):
    positions = kmp(ciphertext, pattern)
    gaps = [
        positions[i + 1] - positions[i]
        for i in range(len(positions) - 1)
    ]

    print()
    print(f"Rank {rank}")
    print("Pattern:", pattern)
    print("Pattern length:", len(pattern))
    print("Positions:", positions)
    print("Gaps:", gaps)

```

키 길이는 특정 패턴의 반복 간격의 공약수로 탐지하는 것이 가능하다. 탐지한 패턴의 간격은 다음과 같다.

패턴	간격	공약수
BKDKFNCNMWKOOWMJME	290	2 5 29
PJTXPQDQRKQWYICSPN	175	5 5 7
MZSWDZTOZNN	140	2 7 2 5
APPUSNJ	55	5 11
JDTJB	115	5 23

공약수는 5가 되어 키 길이는 3보다 크고 5 이하인 값인 5로 추정할 수 있다.

4-나. 키 길이를 $|k|$ 라 할 때, 암호문을 $|k|$ 개의 부분열로 나눈다. 각 부분열은 동일한 단일 문자 치환으로 암호화되어 있다. 각 부분열에 대해 카이제곱 검정 또는 최빈 문자 추정법을 적용하여 각 위치의 키 k 를 구하시오.

키 길이 5로 암호화된 각 암호문을 부분열로 구성할 수 있다. 이러한 부분열을 모아 5개의 그룹을 만들었다.

5개의 그룹으로부터 문제에서 제시된 빈도와의 카이제곱 검정을 통해 각 위치의 키를 얻어낼 수 있었다. 실행 코드는 다음과 같다.

```
import string

upper = string.ascii_uppercase

freq = [
    8.17, 1.50, 2.78, 4.25, 12.70, 2.23, 2.02,
    6.09, 6.97, 0.15, 0.77, 4.03, 2.41, 6.75,
    7.51, 1.93, 0.10, 5.99, 6.33, 9.06, 2.76,
    0.98, 2.36, 0.15, 1.97, 0.07
]

def rotlist(values, count):
    count %= len(values)
    return values[count:] + values[:count]

def chi_square(observed, expected):
    return sum(
        (observed_count - expected_count) ** 2 / expected_count
        for observed_count, expected_count in zip(observed, expected)
    )

def find_key(text):
    freq_c = [text.count(c) for c in upper]
    total = len(text)
    freq_n = [total * percentage / 100 for percentage in freq]
    results = []

    for key in range(26):
        p = rotlist(freq_c, key)
        score = chi_square(p, freq_n)
        results.append((key, score))

    return min(results, key=lambda result: result[1])

with open("ciphertxts2.txt", "r") as f:
    lines = f.readlines()

idx1 = ""
idx2 = ""
idx3 = ""
idx4 = ""
```

```

idx5 = ""

for line in lines:
    text = "".join(c for c in line.upper() if c in upper)
    idx1 = idx1 + text[0::5]
    idx2 = idx2 + text[1::5]
    idx3 = idx3 + text[2::5]
    idx4 = idx4 + text[3::5]
    idx5 = idx5 + text[4::5]

groups = [idx1, idx2, idx3, idx4, idx5]
key_values = []

for number, text in enumerate(groups, 1):
    key, score = find_key(text)
    key_values.append(key)
    print(f"idx{number}: length = {len(text)}, key = {key}, letter = {upper[key]}, chi-square = {score:.3f}")

print("Key values:", key_values)
print("Key:", "".join(upper[key] for key in key_values))

```

실행 결과는 다음과 같았고 각 위치의 키는 다음과 같다.

그룹	복호화 키(알파벳 순서)	검정 결과	암호화 키(알파벳 순서)
idx1	K(10)	222	Q(16)
idx2	L(11)	179	P(15)
idx3	V(21)	406	F(5)
idx4	O(14)	231	M(12)
idx5	J(9)	165	R(17)

4-다. 나에서 추측한 키 값으로 다음 암호문을 복호화하고 평문을 제시하시오.

찾아낸 키 값 KLVOJ로 복호화한 결과는

```

THESOVEREIGNTYOFTHEREPUBLICOFKOREASHALLRESIDEINTHEPEOPLEANDALLSTATEAUTHORITYSH
ALLEMANATEFROMTHEPEOPLE

```

5. 다음 4개의 문장은 시저암호와 비제네르 암호가 혼합되어 있다. 주어진 Ciphertext1.txt와 Ciphertext2.txt를 학습 데이터로하여 AI 분류기 모델을 생성하고, 아래의 암호문이 각각 어떤 유형으로 암호화 되었는지 구분하시오. 이때, 모델 및 데이터 학습 feature를 자세하게 기술하시오.

목표는 카이사르 암호와 비제네르 암호를 판단하는 이진 분류 모델이 되겠다. 암호문 정보를 쪼개서 카이사르 암호화를 적용한 문장과 비제네르 암호를 적용한 문장 2가지로 라벨을 적용하고 지도학습을 통해서 가중치와 편향을 최적화한다. 학습 방식은 로지스틱 회귀를 사용한다.

데이터셋은 Ciphertext1.txt, Ciphertext2.txt로부터 별도로 첨부한 데이터(training.csv)와 같이 설정하였다. 라벨 c는 카이사르 암호, 라벨 v는 비제네르 암호가 된다. 전체 데이터는 카이사르 암호문 157개와 비제네르 암호문 157개로 구성된다.

사용한 학습 모델은 로지스틱 회귀이다. 각 암호문에서 영문 대문자가 아닌 문자를 제거한 다음 일치지수 IC를 계산하고, 이를 모델의 feature인 x 로 사용하였다. 일치지수는 다음 식으로 계산한다. 카이사르 암호의 라벨은 1, 비제네르 암호의 라벨은 0으로 변환하였다. 그리고 각 일치지수에 대해 정규화를 적용한다.
$$x' = \frac{(x - mean)}{std}$$

여기서 mean은 학습 데이터의 일치지수 평균이고, std는 일치지수의 표준편차이다.

정규화된 일치지수에 가중치 w 와 편향 b 를 적용하고, 시그모이드 함수를 통해 카이사르 암호일 확률을 계산하였다. 시그모이드 함수의 출력은 0~1 사이의 확률로 나타난다. 카이사르 암호는 라벨 1, 비제네르 암호는 라벨 0으로 지정하였으므로, 카이사르 암호일 확률이 0.5 이상이면 카이사르 암호로 분류하고 0.5 미만이면 비제네르 암호로 분류하였다.

따라서 모델은 정규화된 일치지수에 가중치와 편향을 적용한 결과가 0 이상이면 카이사르 암호로, 0 미만이면 비제네르 암호로 판단한다. 임계값은 기본 임계값인 0.5를 사용하였다.

학습은 sklearn으로 실행하였으며, StandardScaler와 LogisticRegression을 make_pipeline으로 연결하였다. StandardScaler는 학습 데이터의 일치지수 평균과 표준편차를 계산하여 입력 feature를 표준화하고, LogisticRegression은 표준화된 일치지수를 바탕으로 로그 손실을 최소화하도록 가중치와 편향을 최적화한다. training.csv에 포함된 암호문 전체를 fit() 함수에 입력하여 모델을 학습시켰다. 학습을 완료한 후에는 네 개의 새로운 암호문에 대해서도 동일하게 일치지수를 계산하고, 학습된 모델의 predict()와 predict_proba()를 사용하여 암호 종류와 카이사르 암호일 확률을 출력하였다.

실행 코드와 실제 실행 결과는 다음과 같다.

```
import csv
import string

from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

upper = string.ascii_uppercase

def IC(text):
    text = "".join(c for c in text.upper() if c in upper)
    n = len(text)

    if n < 2:
        return 0

    counts = [text.count(c) for c in upper]
    return sum(count * (count - 1) for count in counts) / (n * (n - 1))

with open("training.csv", "r", newline="") as f:
    rows = list(csv.DictReader(f))

x = [[IC(row["ciphertext"])] for row in rows]
y = [1 if row["label"] == "c" else 0 for row in rows]

model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(x, y)
```

```

texts = [
    "NKRRUZNOYOYGIRGYOIGRIOVNKXGTGREYOYVXUHRKSLUXZKNKIXEVZGTGREYOYIUSVKZOOZOUTIUTMXGZAR
    GZOUTYUTMKZZOTMZNKIUXXXKIZGTYCKX",
    "ROVVYDRSCCKMKVCCSMKVMMSZROBKXKVICSCZBYLVOWPYBDROMBIZDKXKVICSCMYWZODSDSYXMYXQBK
    DEVKDSYXCXQODDSXQDROMYBBOMDKXCGOB",
    "DRKXUIYEPYBIYEBZKBDSMSZKDSYXGOGSCRIYEKVVDRLOCD SXIYEBPEDEBOOXNOKFYBC",
    "ZNGTQEUALUXEUAXVGXZOIOVGZOUTCKCOYNEUAGRRZKNKHYZOTEUAXLAZAXKKTJKGBUXY"
]

```

```

values = [IC(text) for text in texts]
predictions = model.predict(values)
probabilities = model.predict_proba(values)[:, 1]

```

```

for number, text in enumerate(texts, 1):
    value = IC(text)
    label = predictions[number - 1]
    probability = probabilities[number - 1]
    name = "Caesar" if label == 1 else "Vigenere"
    print(f"{number}: IC = {value:.6f}, P(c) = {probability:.6f}, class = {name}")

```

```

1번: IC = 0.063867, P(c) = 0.995223, 카이사르 암호
2번: IC = 0.063867, P(c) = 0.995223, 카이사르 암호
3번: IC = 0.053995, P(c) = 0.694149, 카이사르 암호
4번: IC = 0.053995, P(c) = 0.694149, 카이사르 암호

```

1번과 2번 암호문은 카이사르 암호일 확률이 약 99.52%로 나타났으며, 3번과 4번 암호문은 약 69.41%로 나타났
다. 네 암호문 모두 확률이 기본 임계값인 0.5 이상이므로 모두 카이사르 암호로 분류되었다.